Hide-and-Seek Drone Swarm

CentraleSupélec Multi-Agent Systems — Technical Report

May 27, 2026

Abstract

This report describes the design and implementation of a multi-agent hide-and-seek system in which a single airborne *seeker* drone tries to spot a group of *hider* drones that hide behind obstacles. The system is built on top of the CentraleSupélec Volìere TP framework: every drone controller is a Python function that the simulator (`standalone_sim.py`) calls each tick with the current world state, and must return a velocity command. The contribution of this work lies in the modules around `tp_algos.py`: an obstacle-consensus law with both an asymptotic linear term and a barrier term; an inter-agent dispersion consensus law (gradient flow on the swarm potential $V = (k/2)\sum_{i \neq j} 1/\|p_i - p_j\|$) that maximizes pairwise distances among the hiders; a 3D field-of-view sensor; a shared-memory broadcast channel between hiders; a decentralized *cover-assignment* primitive that solves a min-cost matching so each hider takes a distinct cover face behind an obstacle relative to the last-known seeker pose; and a controller-level 3D inter-drone repulsion that lives inside `tp_algos.py` itself and adds a hard "never closer than 1 m" fence on top of the per-controller behaviours. The overall control law combines a weakened Lissajous exploration drift, a consensus-based obstacle push, the inter-agent dispersion consensus, a top-level 3D drone-to-drone separation, and soft boundary repulsion, with a single magnitude saturation at the actuator limit.

Contents

1 Project context and high-level architecture

The project is a CentraleSupélec TP (2A/3A) on multi-agent systems. The target hardware platforms are Crazyflie 2 nano-quadrotors (acting as hiders) and a DJI RoboMaster TT drone (acting as the seeker), flying inside the school's *Volìere* arena. The pedagogical aim is to use a hide-and-seek game as an accessible illustration of multi-agent control, both for the students implementing it and for the children visiting the Volìere.

Rules of the game.

- A single seeker drone (RMTT) cruises the arena on a slow sweep path. It *never receives the hiders' positions* – it has to find them with vision.
- Several hider drones (CF2) roam the arena. They know the seeker's position only through *their own vision* (a forward cone), and only communicate sightings to each other through a shared broadcast channel.
- When the seeker's forward cone contains a hider *and* the line of sight is not occluded by an obstacle, the hider is "caught" and triggered to land.

Module decomposition. The code base is split into small files that each address one concern:

Module	Role
`standalone_sim.py`	The simulator. Owns the world state, calls each robot's controller in a thread pool, integrates kinematics, detects collisions, renders the 3D figure. *Untouched in this project.*
`tp_algos.py`	The required entry-point file (prof requirement). Beyond the dispatch shims, it hosts a controller-level 3D inter-drone repulsion (`_compute_drone_repulsion_3d`) and applies it on top of both the seeker and hider commands. *Edits kept inside the marked `TO BE MODIFIED` blocks.*
`cf2_milestone1.py`	Top-level hider controller: composes a goal force, obstacle consensus, inter-agent dispersion consensus, and boundary repulsion; switches between roaming and cover-assignment hide mode.
`cf2_sensing.py`	The hider's sensing primitives: intent heading and FoV-cone obstacle sensing on an axis-aligned bounding box (AABB) model.
`cf2_consensus.py`	Two consensus laws: the obstacle push (linear + barrier, used by both hiders and seeker) and the inter-agent dispersion law (Coulomb-form gradient flow, used by the hiders to maximize pairwise distances).
`cf2_hider.py`	Inter-hider communication (shared seeker-sighting memory), single-target cover geometry, and the multi-agent cover-assignment solver (deterministic min-cost matching).
`rmtt_seeker.py`	Seeker controller (`compute_seeker_cmd`, which does *not* accept hider positions) and the `can_see` predicate shared with the referee and the hiders' seeker-detection logic.
`game_referee.py`	Per-frame catch detection using the same `can_see` predicate as the seeker's vision cone.

`sim_viz.py`	Visualization overlays: 3D FoV cones, position trails, status HUD.

Data flow. At each tick $t = k\,\Delta t$ ($\Delta t = 0.05\,\mathrm{s}$), the simulator:

1. Snapshots all pose arrays so the controllers see a consistent world.
2. Submits each robot's controller (the public `tp_algos._controller` entry points) to a thread pool. The controller returns a target velocity (and for drones, a target altitude / takeoff / land flags).
3. Integrates the returned commands with a simple Euler step, applies the actuator-rate saturations, adds a Gaussian position noise term ($\sigma = 5\,\mathrm{mm}$ per step).
4. Calls `game_referee.update_catches` to mark any catches.
5. Runs collision detection (boundary, obstacle, robot-to-robot) and colours the robots' safety globes red on contact.
6. Calls `sim_viz.update` to refresh cones, trails and the HUD.

Default scenario. The simulator ships with `nbRMTT = 1` seeker, `nbCF2 = 4` hiders, and `nbObstacle = 2`: a $1.0 \times 0.5 \times 2.5\,\mathrm{m}$ pillar at $(-0.5, 1.5, 0)$ and a $0.5 \times 1.1 \times 2.5\,\mathrm{m}$ pillar at $(0.8, -2.0, 0)$. The two pillars sit in opposing "rooms" of the arena (positive vs. negative y), keep at least $1.4\,\mathrm{m}$ of clearance from every wall (well above $r_{\text{safe}} = 0.6\,\mathrm{m}$), and are $\sim 3.7\,\mathrm{m}$ apart center-to-center. With four hiders the swarm has roughly enough room to occupy one distinct cover face per drone (the two AABBs jointly expose up to eight far-side faces depending on seeker position), which is precisely what the cover-assignment law in Sec. 3.4 exploits. The RMTT starts at $(-1.5,\,0,\,0)$ – mid-arena on the $-x$ side, between the two obstacles – and is forced into takeoff at $t = 0$. The four CF2 hiders start at the corners of an inset rectangle, $(\pm 1.8,\,\pm 3.8,\,0)$, and take off once their controllers request it. The corner layout was chosen to satisfy four simultaneous constraints at $t = 0$: pairwise hider spacing $\geq 3.6\,\mathrm{m}$ (well above the $1\,\mathrm{m}$ hard floor of Sec. 10), seeker-to-hider distance $\geq 3.82\,\mathrm{m}$ (outside the seeker's $3.5\,\mathrm{m}$ vision range, so no hider is in the seeker's cone at spawn), wall clearance $\geq 0.7\,\mathrm{m}$ (outside the $0.6\,\mathrm{m}$ boundary repulsion margin), and obstacle clearance $\geq 1.35\,\mathrm{m}$ (well above $r_{\text{safe}} = 0.6\,\mathrm{m}$). Each room of the arena – positive and negative y – starts with two hiders, so the cover-assignment law always has candidates on both sides regardless of where the seeker's sweep takes it first.

2 Mathematical model

2.1 State and kinematics

Each drone is modelled as a point mass with state $\mathbf{p}_i = (x_i, y_i, z_i)^\top \in \mathbb{R}^3$ and a velocity command $\mathbf{u}_i = (v_x, v_y, v_z)^\top$. The integration is a forward Euler step with Gaussian noise:

$$\mathbf{p}_i(t + \Delta t) \;=\; \mathbf{p}_i(t) + \mathbf{u}_i \Delta t + \eta_i, \qquad \eta_i \sim \mathcal{N}\big(\mathbf{0}, \sigma^2 \mathbf{I}_3\big)\,, \qquad \sigma = 0.005\,\mathrm{m}.$$

Before integration the simulator applies a 3D magnitude clamp at the actuator limit (`clamp_vel3d` in `standalone_sim.py`).

For CF2 the controller actually returns $(v_x, v_y, z_{\text{dist}})$ where z_{dist} is the *desired altitude*. The simulator converts it to a vertical velocity $v_z = z_{\text{dist}} - z$ and applies the 3D clamp on (v_x, v_y, v_z) together.

2.2 Obstacles

Obstacles are axis-aligned bounding boxes (AABBs) standing on the floor:

$$\mathcal{B}_j \;=\; [o_x - \tfrac{s_x}{2},\, o_x + \tfrac{s_x}{2}] \times [o_y - \tfrac{s_y}{2},\, o_y + \tfrac{s_y}{2}] \times [0,\, s_z].$$

The closest point on $\mathcal{B}_j$ to a query point $\mathbf{p}$ is the per-axis clamp of $\mathbf{p}$ into the box, which collapses to $\mathbf{p}$ itself when $\mathbf{p} \in \mathcal{B}_j$.

3 Hider controller (`cf2_milestone1.py`)

3.1 Force composition

The controller assembles four terms:

$$\mathbf{F}_{\text{total}} \;=\; \mathbf{F}_{\text{goal}} + \mathbf{F}_{\text{obs}} + \mathbf{F}_{\text{disp}} + \mathbf{F}_{\text{bnd}},$$

and outputs the planar component of $\mathbf{F}_{\text{total}}$ after a magnitude saturation at $v_{\max} = 0.45\,\text{m/s}$ (chosen below the hardware cap MAX_V_CF2 $= 0.6\,\text{m/s}$ to leave headroom for the vertical channel inside the simulator's 3D clamp).

Goal attraction

$$\mathbf{F}_{\text{goal}} \;=\; k_g\,(\mathbf{p}_{\text{goal}} - \mathbf{p}), \qquad k_g = 0.15.$$

The goal is either the drone's Lissajous waypoint (Sec. 3.2) or its assigned cover target (Sec. 3.4), depending on whether the swarm is in roaming or hide mode. The small gain (0.15, down from an earlier 0.65) is deliberate: in roam mode it turns the Lissajous into a *slow exploration drift*, so the inter-agent dispersion consensus below becomes the dominant driver and the swarm does not freeze at a fixed orbit the seeker can memorize; in hide mode the same small gain keeps the cover attraction from overwhelming the obstacle barrier near the cover face.

Obstacle consensus

Each sensed obstacle j contributes (see Sec. 4)

$$\mathbf{F}^{(j)}_{\text{obs}} \;=\; \underbrace{k_{\text{obs}}\,\max(0, r_{\text{safe}} - d_j)}_{\text{linear}}\,\mathbf{n}_j \;+\; \underbrace{k_{\text{bar}}\left(\frac{r_{\text{safe}}}{\max(d_j, \varepsilon)} - 1\right)}_{\text{barrier}}\mathbf{n}_j,$$

with d_j the distance to the obstacle surface, $\mathbf{n}_j$ the outward normal (surface $\to$ drone), and $\varepsilon = 0.02\,r_{\text{safe}}$.

Inter-agent dispersion consensus

Per-agent gradient of the swarm potential $V_{\text{disp}}(\mathbf{p}_1, \ldots, \mathbf{p}_N) = \frac{k_{\text{disp}}}{2}\sum_{i \neq j} 1/\|\mathbf{p}_i - \mathbf{p}_j\|$ in the xy plane:

$$\mathbf{F}^{(i)}_{\text{disp}} \;=\; -\nabla_{\mathbf{p}_i} V_{\text{disp}} \;=\; k_{\text{disp}} \sum_{j \neq i} \frac{\mathbf{p}_i - \mathbf{p}_j}{\|\mathbf{p}_i - \mathbf{p}_j\|^3}, \qquad k_{\text{disp}} = 0.4.$$

This is a true multi-agent consensus law on the aggregate cost V_{disp}: every hider's local law decreases the *same* scalar function along the joint gradient flow, so the swarm collectively minimizes V_{disp}, equivalently maximizes inverse pairwise distances, equivalently spreads as far apart as the boundary and obstacle constraints allow. Unlike a short-range repulsion, the law acts at *all* distances – every hider always knows about every other hider's pull. Symmetry implies the centroid of the swarm is invariant under $\mathbf{F}_{\text{disp}}$ alone (the goal, obstacle, and boundary terms then steer the centroid). A numerical floor $\varepsilon_{\text{disp}} = 0.05\,\text{m}$ on the denominator avoids a blow-up if two hiders briefly co-locate. A stricter 3D "never closer than 1 m" separation between *all* flying drones (hiders *and* seeker) is added on top in `tp_algos.py` (Sec. 10).

Boundary repulsion

Each wall at distance d inside a margin $m = 0.6\,\text{m}$ pushes inward:

$$F^{(w)}_{\text{bnd}} = g_{\text{bnd}} \left(\frac{1}{\max(d, 10^{-4})} - \frac{1}{m} \right), \qquad g_{\text{bnd}} = 0.09.$$

The gain is intentionally small (0.09) – the goal is a gentle nudge, not a hard bounce.

3.2 Lissajous roaming path

Each hider i tracks a 3D Lissajous waypoint

$$\mathbf{p}^{(i)}_{\text{goal}}(t) = \begin{pmatrix} c_x + A_x \sin(\omega^{(i)}_x t + \varphi^{(i)}_x) \\ c_y + A_y \sin(\omega^{(i)}_y t + \varphi^{(i)}_y) \\ c_z \end{pmatrix},$$

with $A_x = 1.6$, $A_y = 3.0$, $c_z = 1.0$, and per-axis, per-drone angular frequencies

$$\omega^{(i)}_x = 0.16 + 0.018\,(i-1), \quad \omega^{(i)}_y = 0.11 + 0.015\,(i-1), \quad \omega^{(i)}_z = 0.09 + 0.013\,(i-1)\ \text{rad/s},$$

and coprime phase offsets $\varphi^{(i)}_x = (i-1)\cdot 2\pi/3$, $\varphi^{(i)}_y = (i-1)\cdot 2\pi/5$, $\varphi^{(i)}_z = (i-1)\cdot 2\pi/7$. The three axes use different base frequencies (not a single $0.16+0.018(i-1)$) so a single drone alone already traces a non-degenerate Lissajous, and the small per-drone increments stop the swarm from collapsing onto a shared orbit.

Two intentionally different "Lissajous targets". There are *two* versions of the Lissajous formula in the code base, and they differ on purpose:

- `cf2_milestone1._lissajous_target` (used as the position goal for $\mathtt{F}_{\text{goal}}$) returns $t_z = c_z$, i.e. the $A_z \sin(\cdot)$ term is commented out so the drone holds a constant cruise altitude and the on-screen cone stays at a predictable height.
- `cf2_sensing.intent_heading` (used to build the FoV-cone *axis*) returns $t_z = c_z + A_z \sin(\omega^{(i)}_z t + \varphi^{(i)}_z)$, $A_z = 0.5$, so the cone gets a gentle vertical sweep – handy if the seeker is at a slightly different altitude.

Position tracking is altitude-locked, but *sensing* is not, which keeps detection ranges 3D without forcing the controller to chase z.

3.3 Mode switching: roam vs. hide

The hiders communicate seeker sightings through a single module-level dict in `cf2_hider.py`. The mode logic in `compute_roaming_cmd` is:

1. Compute the FoV-based seeker detection: if `can_see(self, seeker, intent_heading, FOV_HALF_ANGLE = 60°, range = 3.0m)`, call `report_seeker_sighting(seeker_pos, t)`.
2. Read the shared memory via `get_last_seen(t)`. If a sighting is fresher than TTL = 5 s, switch the goal target to the cover point (next section); otherwise, use the Lissajous target.

The structural asymmetry is important: the seeker controller never reads or writes the shared memory, so "hiders can talk to each other but the seeker can't listen" is enforced by interface, not by convention.

3.4 Cover-assignment geometry (multi-agent)

Hide mode is a small *cooperative task-allocation* problem: there are several cover spots and several hiders, and the swarm wants every hider on a distinct spot so the seeker cannot harvest them in one cone sweep. The solution lives in `cf2_hider.assign_cover_targets` and runs in three steps.

1. Candidate enumeration. For every obstacle (center $\mathbf{o}_j$, half-extents (h_x, h_y)) and every AABB face with outward normal $\hat{\mathbf{n}} \in \{\pm\hat{\mathbf{e}}_x, \pm\hat{\mathbf{e}}_y\}$, build the candidate

$$\mathbf{c}_{j,\hat{\mathbf{n}}} \;=\; \underbrace{\mathbf{o}_j + (\hat{\mathbf{n}}\cdot\mathbf{h})\,\hat{\mathbf{n}}}_{\text{face center}} + d_{\text{cov}}\,\hat{\mathbf{n}}, \qquad d_{\text{cov}} = 0.8\,\text{m},$$

where $\mathbf{h} = (h_x, h_y)$. The face is *kept* only if $\hat{\mathbf{n}}\cdot(\mathbf{o}_j - \mathbf{s}) > 0$, i.e. the normal points away from the seeker – only those faces actually occlude line of sight to the cover point. With two obstacles this yields up to eight candidates. The $d_{\text{cov}} > r_{\text{safe}}$ inequality is deliberate: the candidate sits *outside* the obstacle-consensus push zone, so a hider can come to rest on it without fighting the avoidance law.

2. Cost matrix. With N hiders and $M \geq N$ candidates, define $C_{ij} = \|\mathbf{p}_i^{xy} - \mathbf{c}_j\|_2^2$. Squared distance gives the same min-cost solution as Euclidean distance and avoids $N \cdot M$ square roots.

3. Min-cost injective assignment. Find $\pi : \{1, \ldots, N\} \to \{1, \ldots, M\}$ injective minimizing $\sum_i C_{i,\pi(i)}$. For the default scenario $(N, M) \leq (4, 8)$ the search space is $P(M, N) \leq 1680$ permutations, so a branch-and-bound brute force is faster and dependency-free compared to importing SciPy's `linear_sum_assignment`. The implementation prunes any partial assignment whose accumulated cost already exceeds the best complete one.

Decentralization. The simulator hands every hider the same `cf2_poses` matrix and the same shared seeker sighting; `assign_cover_targets` is deterministic in those inputs. So every hider independently computes the *same* matching and reads its own row from the result – a consensus reached by "every node solves the same optimization" rather than by message exchange. No explicit communication is needed beyond the existing shared-memory sighting channel.

Fallback. If $M < N$ (one obstacle, or the seeker hides most far-side faces), the $N - M$ leftover hiders fan out radially around the seeker at distance 1.5 m on angles $2\pi k/(N - M)$. They no longer get a true occlusion but they no longer pile up either, and the dispersion consensus keeps pushing them apart in the meantime. The single-target `seek_cover_target` function (older nearest-cover-only behaviour) is kept exported for backward compatibility and tests, but is no longer on the hot path.

4 Consensus laws (`cf2_consensus.py`)

The module hosts two consensus laws used by the hiders. The obstacle consensus (Sec. 4.1) is also used by the seeker. The inter-agent dispersion law (Sec. 4.2) is hider-only and is what gives the swarm its coordinated "spread out" behaviour.

4.1 Obstacle consensus

Definition

Given a list of sensed obstacles, each with closest-surface distance d_j and outward normal $\mathbf{n}_j$, the consensus command is

$$\mathbf{u} \;=\; \sum_{j\,:\,d_j<r_{\text{safe}}} \left[k_{\text{obs}}\,(r_{\text{safe}}-d_j)+k_{\text{bar}}\left(\frac{r_{\text{safe}}}{\max(d_j,\varepsilon)}-1\right)\right]\mathbf{n}_j.$$

The function does *not* saturate; saturation is the caller's responsibility (the hider controller applies the magnitude cap on the summed force, after adding the goal/separation/boundary contributions).

Properties

Asymptotic (linear term). Define $e_j(\mathbf{p})=\max(0,r_{\text{safe}}-d_j(\mathbf{p}))$ and the Lyapunov candidate

$$V(\mathbf{p}) \;=\; \tfrac{1}{2}\sum_j e_j(\mathbf{p})^2.$$

For static obstacles, $\nabla d_j = \mathbf{n}_j$ on the violating set, so the purely-linear closed loop $\dot{\mathbf{p}} = -k_{\text{obs}}\sum_j e_j\mathbf{n}_j$ satisfies $\dot{V} = -k_{\text{obs}}\sum_j e_j^2 \le 0$, with equality only when all violations vanish. The drone therefore converges to the safe set $\{\mathbf{p} : d_j(\mathbf{p}) \ge r_{\text{safe}}\ \forall j\}$ – an asymptotic consensus on "be outside every safety bubble".

Bounded actuation (barrier term). The linear term alone has bounded magnitude on bounded violations, so a strong enough goal-attraction force could overpower it close to the obstacle. The barrier term grows like r_{safe}/d_j as $d_j \to 0$, which *always* dominates any bounded attraction once the drone is close enough. Combined with the actuator saturation in the caller, the closed-loop velocity is bounded everywhere while the dynamics inside $\{d_j < r_{\text{safe}}\}$ remain dominated by the avoidance push.

Numerical safety. The denominator uses $\max(d_j,\varepsilon)$ with $\varepsilon = 0.02\,r_{\text{safe}} = 12\,\text{mm}$. In simulation a drone may briefly penetrate an obstacle due to the discrete integration step; this clamp keeps the law finite while still producing a very large push.

4.2 Inter-agent dispersion consensus

Definition

For the hider swarm $\{\mathbf{p}_1,\ldots,\mathbf{p}_N\} \subset \mathbb{R}^2$ (only the planar component enters the law), define the aggregate cost

$$V_{\text{disp}}(\mathbf{p}_1,\ldots,\mathbf{p}_N) \;=\; \frac{k_{\text{disp}}}{2}\sum_{\substack{i,j\\ i\neq j}} \frac{1}{\|\mathbf{p}_i-\mathbf{p}_j\|}.$$

Each hider runs the per-agent gradient descent

$$\mathbf{u}_i \;=\; -\nabla_{\mathbf{p}_i}V_{\text{disp}} \;=\; k_{\text{disp}}\sum_{j\neq i}\frac{\mathbf{p}_i-\mathbf{p}_j}{\|\mathbf{p}_i-\mathbf{p}_j\|^3}.$$

This is the Coulomb form: agent i feels an inverse-square push (in *magnitude* per unit charge) from every other agent. The implementation floors $\|\mathbf{p}_i-\mathbf{p}_j\|$ at $\varepsilon_{\text{disp}} = 0.05\,\text{m}$ to keep the gradient bounded if two hiders briefly co-locate.

Properties

Aggregate Lyapunov. V_{disp} is itself a Lyapunov function for the joint flow $\dot{\mathbf{p}}_i = \mathbf{u}_i$ on the unconstrained domain $\{\mathbf{p}_i \neq \mathbf{p}_j \ \forall i \neq j\}$:

$$\dot{V}_{\text{disp}} \;=\; \sum_i \nabla_{\mathbf{p}_i} V_{\text{disp}} \cdot \dot{\mathbf{p}}_i \;=\; -\sum_i \|\nabla_{\mathbf{p}_i} V_{\text{disp}}\|^2 \;\leq\; 0,$$

with equality only at critical configurations of V_{disp}. Every hider's local descent decreases the *same* swarm-level scalar – the defining property of a consensus law on an aggregate cost.

Long-range coupling. There is no cutoff: every other hider always contributes. Contrast with a short-range repulsion (e.g. $1/d^2$ inside a 0.9 m influence radius and zero outside): such a law is silent the moment the swarm is more than 0.9 m apart, so it cannot push two well-separated hiders further apart. The Coulomb form pushes at any distance, so the equilibrium is determined by the boundary and obstacle constraints, not by the law going silent.

Centroid invariance and bounded equilibria. $\mathbf{F}_{\text{disp}}^{(j\to i)} = -\mathbf{F}_{\text{disp}}^{(i\to j)}$ (Newton's third law in form), so the swarm centroid is invariant under $\mathbf{F}_{\text{disp}}$ alone. Bounded equilibria emerge from the combination with the boundary repulsion of Sec. 3.1: the walls confine the swarm to the arena while $\mathbf{F}_{\text{disp}}$ pushes the hiders to the configuration with maximum pairwise distances inside that confinement.

Decentralization. The law uses only $\mathbf{p}_i$ and $\{\mathbf{p}_j\}_{j\neq i}$ – the simulator-provided pose matrix is the only neighbourhood query needed, no extra messaging. In a real deployment the assumed information flow is "each hider broadcasts its position to all the others at controller frequency". The shared-memory channel already exists for seeker sightings (Sec. 3.4) and would extend trivially to position broadcasts.

5 Sensing (`cf2_sensing.py`)

5.1 Intent heading

The heading used both for the FoV cone and the sensing axis is the unit vector toward the current Lissajous target:

$$\hat{\mathbf{h}}(t) \;=\; \frac{\mathbf{p}_{\text{goal}}(t) - \mathbf{p}(t)}{\|\mathbf{p}_{\text{goal}}(t) - \mathbf{p}(t)\|}.$$

Compared to a path-tangent heading $\dot{\mathbf{p}}_{\text{goal}}/\|\dot{\mathbf{p}}_{\text{goal}}\|$, this points where the drone wants to be rather than where the path is currently moving. The practical consequence: when an obstacle is between the drone and its waypoint, the cone is already aimed at the obstacle.

5.2 FoV-cone obstacle sensing

For each obstacle $\mathcal{B}_j$, the function `sense_obstacles`:

1. Computes the closest box point $\mathbf{c}_j$ to the drone (per-axis clamp).
2. Computes the 3D distance $d_j = \|\mathbf{c}_j - \mathbf{p}\|$ and rejects the obstacle if $d_j >$ `sensing_radius`.
3. Applies the angular gate $\hat{\mathbf{h}} \cdot \hat{\mathbf{e}}_j \geq \cos\theta_{\text{fov}}$ where $\hat{\mathbf{e}}_j = (\mathbf{c}_j - \mathbf{p})/d_j$, returning the obstacle's $\mathbf{n}_j = -\hat{\mathbf{e}}_j$ and d_j.

A degenerate case is handled explicitly: if $d_j < 10^{-6}$ (drone is on or inside the obstacle) the function flags it as in-cone with $\mathbf{n}_j = -\hat{\mathbf{h}}$, so the downstream consensus law pushes the drone back along the inverse of its heading.

Why hiders sense omnidirectionally. The hider controller passes $\theta_{\text{fov}} = \pi$ to `sense_obstacles`, effectively disabling the angular gate. The forward 60° cone is reserved for *seeker detection*, not obstacle avoidance, since otherwise a hider would be free to enter an obstacle from a blind side.

6 Seeker controller (`rmtt_seeker.py`)

6.1 Sweep path

The seeker tracks a slow Lissajous sweep:

$$\mathbf{p}^S_{\text{goal}}(t) = \begin{pmatrix} A^S_X \sin(\Omega_X t) \\ A^S_Y \sin(\Omega_Y t + \pi/2) \\ c^S_z + A^S_Z \sin(\Omega_Z t) \end{pmatrix},$$

with $A^S_X = 1.8$, $A^S_Y = 3.5$, $A^S_Z = 0.4$, $c^S_z = 1.0$, and angular frequencies $\Omega_X = 0.18$, $\Omega_Y = 0.13$, $\Omega_Z = 0.08$ rad/s (cycle times $\approx 35, 48, 79$ s). The $\pi/2$ phase between x and y produces a figure-of-eight footprint that sweeps both "rooms" of the arena every cycle.

6.2 Control law

The seeker uses the same consensus law as the hiders (with the same parameters $r_{\text{safe}} = 0.6$, $k_{\text{obs}} = 2$, $k_{\text{bar}} = 0.5$) plus a goal attraction $\mathbf{F}^S_{\text{goal}} = k^S_g(\mathbf{p}^S_{\text{goal}} - \mathbf{p})$, $k^S_g = 0.6$. Saturation is applied separately on the planar magnitude (at $v^S_{\max,xy} = 0.55$ m/s) and on the vertical component (at $|v^S_z| \le 0.35$ m/s).

The signature `compute_seeker_cmd(robot_no, robot_pose, obstacle_pose, obstacle_size, clock)` deliberately does *not* accept hider positions: the seeker's *navigation* cannot use any hider state. The `tp_algos.rmtt_control_fn` wrapper does receive `cf2_poses` (its signature is fixed by the TP framework) and uses them *only* to add the 3D safety-fence repulsion described in Sec. 10 – it never forwards them to `compute_seeker_cmd`, so the controller's planning logic remains hider-blind.

6.3 Search cone and line-of-sight

The search cone has half-angle $\theta_{\text{seek}} = 30°$ and range 3.5 m – narrower but longer than the hiders' cone. The predicate `can_see(apex, target, heading, fov_half, range, obstacles)` returns `True` when:

1. The 3D distance $\|\mathbf{t} - \mathbf{a}\| \le$ range.
2. The cone condition holds: $(\mathbf{t} - \mathbf{a}) \cdot \hat{\mathbf{h}} \ge \cos\theta_{\text{fov}} \|\mathbf{t} - \mathbf{a}\|$.
3. No obstacle box intersects the line segment from apex to target.

The occlusion test uses the standard Kay–Kajiya slab algorithm (`_segment_intersects_aabb`): for each of the 3 axes, compute the parameter interval $[t_1, t_2]$ over which the segment lies between the slab faces; intersect the three intervals; the segment hits the box iff the intersection overlaps $[0, 1]$. Axis-parallel segments are handled with an explicit "inside the slab?" check.

7 Inter-hider communication (`cf2_hider.py`)

The shared state is a single module-level dict:

```
_last_seen = {'position': None, 'timestamp': None}
SEEKER_MEMORY_TTL = 5.0   # seconds
```

Writers (report_seeker_sighting) overwrite both fields atomically; readers (get_last_seen) compare t - timestamp against the TTL and return either the position or None. Last-write-wins is fine here because the question is "what is the most recent fix on the seeker", and between two sightings the older one offers no extra information.

TTL choice. 5 s is enough to keep the swarm in hide mode for several controller cycles after a sighting (the cover-assignment step is itself updated every tick, so the swarm reshuffles quickly inside the window), yet much shorter than a full seeker sweep, so the hiders return to dispersion-driven roaming soon after the threat leaves sight. The exact value is not critical: a shorter TTL biases the swarm toward exploration (more dispersion, less time spent in cover), a longer one biases it toward defence.

Threading. The simulator runs each controller in a ThreadPoolExecutor. Python's GIL plus the atomic-dict-write convention is enough here: a reader can never see a half-written record, only a fully old or fully new one.

8 Catch detection (game_referee.py)

The referee owns a set of caught hider IDs and a dict of catch times. Each frame it iterates over (flying seekers) × (flying, not-yet-caught hiders), calls can_see (the same predicate the seeker would compute internally if it had one), and adds the hider to the caught set if visible. Once caught, a hider stays caught for the rest of the simulation. The hider controller in tp_algos.cf2_control_fn reads this flag via is_caught(robot_no) and returns trigger_land = True, which puts the drone into landing state.

9 Visualization (sim_viz.py)

The visualization layer adds three things on top of the simulator's basic markers:

- **3D FoV cones.** For each flying drone, a triangle-fan polyhedron is built with apex at the drone and axis along its heading. The 16-segment base disk is constructed in the plane perpendicular to the heading using an orthonormal basis built from an arbitrary reference vector (world-z, switching to world-x when the heading is nearly vertical).
- **Position trails.** Each hider keeps a bounded deque (TRAIL_LEN = 80) of recent positions, drawn as a thin green line. When the hider returns to idle (after a catch landing) the trail is cleared so the next takeoff starts fresh.
- **HUD.** A small monospace text in the upper left shows t, the catch count, and an alert on/off flag derived from get_last_seen(t).

The cone colour for the hiders switches from limegreen (roam) to gold (alert) when a fresh sighting is in shared memory, so the global mode is visible at a glance.

10 Simulator hooks and controller-level separation (tp_algos.py)

The TP framework requires every controller to live in tp_algos.py with a fixed signature. Per the project rules, only the body marked TO BE MODIFIED may be edited. Two responsibilities live here:

1. Dispatch each robot type to its dedicated module (cf2_milestone1, rmtt_seeker, . . .).

2. Apply a controller-level 3D drone-to-drone separation *on top of* what each module returns. This is the only place in the code base that has simultaneous access to *both* hider and seeker poses, so it is the natural place to enforce a global "no two drones get closer than 1 m" rule.

10.1 3D inter-drone barrier: a hard 1 m floor

`_compute_drone_repulsion_3d(self_pose, neighbors, min_dist=1.0, activation=1.5)` enforces a *hard* minimum separation $d_{\min} = 1\,\mathrm{m}$ between any pair of drones (hider-hider and hider-seeker alike). For each neighbor k with $d_{ik} < d_{\mathrm{act}}$:

$$\Delta \mathbf{v}_i = g_{\mathrm{rep}} \sum_{k\,:\,d_{ik}<d_{\mathrm{act}}} \left(\frac{1}{d_{ik} - d_{\min}} - \frac{1}{d_{\mathrm{act}} - d_{\min}} \right) \frac{\mathbf{p}_i - \mathbf{p}_k}{\|\mathbf{p}_i - \mathbf{p}_k\|}, \quad g_{\mathrm{rep}} = 30,\ d_{\mathrm{act}} = 1.5\,\mathrm{m}.$$

The magnitude vanishes continuously at $d = d_{\mathrm{act}}$ and diverges as $d \to d_{\min}^{+}$. If a pathological state $d \le d_{\min}$ is ever observed (e.g. at spawn), the function returns a 10^6-magnitude outward command which, after the simulator's per-axis clamp at MAX_V $= 0.6\,\mathrm{m/s}$, drives the drone at full speed purely outward.

Discrete-time non-penetration argument. Let $\Delta t = 0.05\,\mathrm{s}$ and $v_{\max} = 0.6\,\mathrm{m/s}$. The largest closure two drones can produce in one tick is $2v_{\max}\Delta t = 0.06\,\mathrm{m}$. Suppose at tick k we have $d_k > d_{\min}$.

- If $d_k \ge d_{\mathrm{act}}$, the barrier is zero on this step but $d_{k+1} \ge d_k - 0.06 \ge d_{\mathrm{act}} - 0.06 = 1.44\,\mathrm{m} > d_{\min}$.
- If $d_{\min} < d_k < d_{\mathrm{act}}$, the barrier magnitude already exceeds the controllers' bounded xy commands ($\le 0.45\,\mathrm{m/s}$ for the hider, $\le 0.55\,\mathrm{m/s}$ for the seeker) once $d_k \le d_{\mathrm{act}} - 0.06$. The clamp therefore resolves the net velocity to $v_{\max}$ purely outward on both drones, and $d_{k+1} \ge d_k + 2v_{\max}\Delta t > d_{\min}$.

Combining the two cases, $d_k > d_{\min} \Rightarrow d_{k+1} > d_{\min}$ for every k. The 1 m floor is invariant under the closed-loop dynamics.

A numerical degenerate case ($d_{ik} < \varepsilon = 10^{-6}\,\mathrm{m}$) returns a fixed $+x$ push so the function never divides by zero. The sum operates in all three axes, contrary to the planar dispersion consensus in `cf2_milestone1` which only sees xy; the 3D component matters when a hider and the seeker happen to be vertically aligned (which the dispersion law ignores by construction). The two laws compose by addition, so the barrier acts as a short-range 3D hard fence on top of whatever long-range xy geometry the dispersion law has chosen.

10.2 How the helper is used

- `cf2_control_fn`: once the hider is flying and is not caught, it calls `compute_roaming_cmd` for $(v_x, v_y, z_{\mathrm{dist}}, \mathrm{led})$. It then collects *all other* flying CF2s *and all* RMTT seekers as neighbors, asks `_compute_drone_repulsion_3d` for $(\Delta v_x, \Delta v_y, \Delta v_z)$, and adds the result to the controller's (v_x, v_y) and to a synthesized $v_z = z_{\mathrm{dist}} - z$. The corrected v_z is then folded back into z_{dist} so the simulator's "$v_z = z_{\mathrm{dist}} - z$" convention is preserved.
- `rmtt_control_fn`: gets $(v_x, v_y, v_z, \mathrm{led})$ from `compute_seeker_cmd` (which itself never sees hider poses), then collects all flying CF2s as neighbors and adds the 3D repulsion to (v_x, v_y, v_z). This is the one place where the seeker controller layer actually reads `cf2_poses` – but only to keep its distance, not to chase. The asymmetry of the game (the seeker has to *see* the hiders to catch them) is still preserved because the catch logic in `game_referee` uses vision alone.
- `tb3B_control_fn`, `tb3W_control_fn`, `rmep_control_fn`: placeholder goal-seek behaviour ($v_x, v_y = 0.2(g - p)$ toward a hard-coded g). These robots are not part of the current hide-and-seek configuration (`nbTb3B` = `nbTb3W` = `nbRMEP` = 0).

10.3 Hider takeoff handshake

The CF2 takeoff is split between the simulator's state machine and a small per-controller cache in `cf2_control_fn`. The cache (`_takeoff_done_by_robot`, indexed by `robot_no`) starts at `False`; while the drone sits at $z < 0.05\,\mathrm{m}$ it returns `trigger_takeoff = True` and shows a blue LED. The simulator then runs the 3 s linear takeoff (state 1) up to `CF2_HOVER_Z`; once the controller sees $z > 0.1\,\mathrm{m}$ it flips the cache to `True` and from then on delegates to `compute_roaming_cmd`. If the referee marks the hider caught, the wrapper sends `trigger_land = True` and lights the LED red.

The lazy-import trick (`if not hasattr(..., "_cached"): import ...`) keeps the module fast to reload and avoids polluting the framework's expected namespace.

11 Simulation infrastructure (`standalone_sim.py`)

11.1 World and hardware specs

Constant	Value	Meaning
`dt`	0.05 s	Simulation step.
`X_MIN, X_MAX`	±2.5 m	Arena bounds (mirrored in `cf2_milestone1`).
`Y_MIN, Y_MAX`	±4.5 m	
`Z_MIN, Z_MAX`	0, 3.5 m	
`MAX_V_CF2`	0.6 m/s	3D speed cap for hider drones.
`MAX_V_RMTT`	0.6 m/s	3D speed cap for the seeker.
`RAD_CF2`	0.05 m	CF2 safety glob is 2× this.
`RAD_RMTT`	0.08 m	RMTT safety glob is 2× this.
`CF2_HOVER_Z`	1.0 m	Target hover altitude after takeoff.
`RMTT_HOVER_Z`	0.8 m	
`TAKEOFF_TIME`	3.0 s	Linear takeoff over $[0, \mathrm{HOVER\_Z}]$.
`LANDING_TIME`	3.0 s	
`DRONE_POS_NOISE_STD`	0.005 m	Per-step Gaussian noise on each axis.

11.2 State machine

Each drone has a 4-state machine:

$$0\ \text{Idle} \xrightarrow{\text{trigger\_takeoff}} 1\ \text{Takeoff} \xrightarrow{\text{timer expired}} 2\ \text{Flying} \xrightarrow{\text{trigger\_land}} 3\ \text{Landing} \xrightarrow{\text{timer/altitude}} 0.$$

In states 1 and 3 the altitude is driven linearly by the takeoff/landing timer; in state 2 the controller's (v_x, v_y, v_z) are applied with the actuator saturation and noise step.

11.3 Async controller execution

The simulator runs each controller in a `ThreadPoolExecutor` with `max_workers = 20`. The helper `get_async_cmd` submits a job if none is running for the given robot, and otherwise returns the cached last command. This means a controller can block (e.g. `time.sleep`) without stalling the simulation – the drone simply keeps its last command. The trade-off is that controllers can be running on slightly stale snapshots, but in practice the world barely changes in one `dt`.

11.4 Collision detection

Each robot has a "safety glob" of radius 2 × base that turns red on contact. Three checks per frame:

- **Boundary**: arena walls in x, y; ceiling at $z = Z_{\max}$ always; floor at $z = 0$ only for drones in the flying state.
- **Obstacle**: closest-point distance against each AABB (same per-axis-clamp formula as the sensing module).
- **Robot-to-robot**: 3D distance against the sum of the two globs' radii.

Warnings are rate-limited to one per (robot, kind) per second.

12 Tunables summary

Name	Value	Where / why
r_{safe}	0.6 m	Desired clearance from obstacles (`R_SAFE` in both hider and seeker).
k_{obs}	2 (1/s)	Linear consensus gain.
k_{bar}	0.5 m/s	Barrier gain near contact.
ε	$0.02\,r_{\text{safe}}$	Numerical clamp inside the barrier.
θ_{fov} hiders	60°	Forward cone for seeker detection and visualization.
sensing radius	2.0 m	Obstacle sensing range (hider and seeker).
seeker detection range	3.0 m	Hider-side range to detect the seeker.
k_{disp}	0.4	Inter-agent dispersion gain (`cf2_milestone1.K_DISP`); scale of the Coulomb-form swarm push.
$\varepsilon_{\text{disp}}$	0.05 m	Pairwise-distance floor inside the dispersion law (numerical safety).
k_g	0.15	Goal attraction gain in the hider controller (weakened so dispersion dominates).
$d_{\min}$	1.0 m	3D drone-to-drone hard floor; mathematically guaranteed invariant under the closed-loop dynamics (`tp_algos.DRONE_MIN_DISTANCE`).
d_{act}	1.5 m	Activation buffer above $d_{\min}$ where the barrier turns on; sized so $d_{\text{act}} - d_{\min} \gg 2v_{\max}\Delta t = 0.06$ m (`tp_algos.DRONE_ACTIVATION_DISTANCE`).
g_{rep}	30.0	3D drone-to-drone barrier gain (`tp_algos.DRONE_REPULSION_GAIN`).
boundary margin	0.6 m	Distance at which wall push turns on.
g_{bnd}	0.09	Wall push gain.
$v_{\max}^{\text{xy}}$ hider	0.45 m/s	Final xy saturation, < MAX_V_CF2.
Z rate limit	0.3 m	Per-call $\lvert z_{\text{dist}} - z\rvert$ cap.
d_{cov}	0.8 m	Cover offset past the obstacle surface.
TTL	5 s	Lifetime of a seeker sighting in shared memory.
θ_{seek}	30°	Seeker FoV half-angle.
seeker vision range	3.5 m	Seeker FoV slant length.
$\Omega_X^S, \Omega_Y^S, \Omega_Z^S$	0.18, 0.13, 0.08 rad/s	Seeker Lissajous.
A_X^S, A_Y^S, A_Z^S	1.8, 3.5, 0.4 m	Seeker Lissajous.

$\omega_x^{(i)}$	$0.16 + 0.018(i-1)$ rad/s	Hider Lissajous (per-drone, x).
$\omega_y^{(i)}$	$0.11 + 0.015(i-1)$ rad/s	Hider Lissajous (per-drone, y).
$\omega_z^{(i)}$	$0.09 + 0.013(i-1)$ rad/s	Hider Lissajous (per-drone, z; used in `intent_heading` only).
A_x, A_y	1.6, 3.0 m	Hider Lissajous amplitudes in xy.
A_z	0.5 m	Used only in `intent_heading` (FoV axis z); disabled in the position target.
TRAIL_LEN	80 frames	Trail length in the visualization.

13 Discussion and design choices

Why separate the FoV cone from the obstacle sensor. The first iteration of the hider used the same 60° cone for both seeker detection and obstacle avoidance. The result: the drone would happily back into a wall when its heading was pointing forward. Splitting the two (omnidirectional for obstacles, forward cone for vision) is what makes the hider safe in a cluttered arena while still "looking somewhere".

Why a barrier on top of the linear consensus. A pure linear law with the same k_{obs} would let the goal-attraction push the drone arbitrarily close to an obstacle if the goal is on the far side. The barrier term grows like r_{safe}/d, so it eventually dominates any *bounded* attraction; combined with the magnitude saturation at the actuator limit, this gives "always-bounded velocity, never collides" under reasonable goals.

Why cover behind the surface, not the center. For elongated obstacles, a cover point at distance r_{cov} from the obstacle center can still be inside the AABB. The implementation computes the parametric ray-AABB exit and adds the offset *past* the surface; combined with $d_{\text{cov}} > r_{\text{safe}}$ this gives a cover point that sits in the safe set, so the hider can rest there without being pushed away.

Why a gradient flow (Coulomb potential) for inter-hider dispersion. The earlier implementation used a short-range inverse-square push with a 0.9 m cutoff – effective for collision avoidance but invisible beyond that radius, so the swarm could perfectly well sit on four parallel Lissajous orbits and offer the seeker a tightly-correlated target. Replacing it with $\mathbf{u}_i = -\nabla_{\mathbf{p}_i} V_{\text{disp}}$ on the Coulomb potential gives *(i)* a single aggregate cost that all agents collectively minimize – the textbook multi-agent consensus signature *(ii)* a force that never goes silent, so any contraction of the swarm is met by an immediate restoring push, and *(iii)* a closed-form Lyapunov certificate. The price is the equilibrium configuration depends on the boundary repulsion, which is why g_{bnd} stays small (0.09): strong walls would push the swarm into the centre and undo the dispersion.

Why an explicit cover assignment, not nearest-cover-per-agent. With four hiders and two obstacles, the "each hider runs to its closest cover" policy frequently sends two hiders to the same face. Once they arrive, the dispersion push spreads them but they remain on the same *visible* side of the obstacle – a single seeker sweep is enough to catch both. Solving a min-cost matching makes the assignment a global property of the swarm, and the candidate set is built per-face (not per-obstacle) so the granularity matches the number of hiders. Decentralized execution falls out of determinism: every hider computes the same optimization on the same inputs and reads its own row.

Why hide the hider positions from the seeker controller specifically. The *planning* module of the seeker, `compute_seeker_cmd`, does not accept `cf2_poses`. The `tp_algos` wrapper still has access to them (its signature is dictated by the framework) and uses them only for the 3D safety-fence repulsion. The split is deliberate: a safety fence is a property of the world, not of the game (we do not want drones colliding mid-air); whereas the search behaviour must remain vision-only, which is what the referee enforces via `can_see`. Future changes to the seeker's search logic would have to explicitly thread hider positions through `compute_seeker_cmd`'s signature, which would be visible in review.

Why shared-memory comms rather than message passing. The simulator already runs each controller in a thread; a single last-write-wins dict is the smallest faithful model of "every hider can hear every other hider's report". A more elaborate scheme (per-pair channels, ack/nak, etc.) would not change the closed-loop behaviour because the controllers only care about "most recent fix on the seeker".

14 Future work

- **Multiple seekers and adversarial cooperation.** The `can_see` predicate is already multi-seeker aware (the referee loops over all flying RMTTs). What is missing is a seeker coordination law that splits the arena and avoids redundant sweeps.
- **Bounded consensus with a soft centroid anchor.** The inter-agent dispersion law is purely repulsive; bounded equilibria currently come from the arena walls. Adding a weak attractive term toward a moving centroid (or toward an assigned region centroid in a Voronoi tessellation) would give a true bounded-consensus behaviour and would let the swarm hold formation away from the walls.
- **Per-hider altitude diversity.** The A_z term is already threaded through `intent_heading` but stripped from `_lissajous_target`, so the FoV cone wobbles in z while the position is altitude-locked. Re-enabling the A_z component in the position target would spread the hiders vertically, making the seeker's narrow cone less effective; the Z rate limit ($0.3\,\mathrm{m/call}$) is already small enough that this would not destabilize the actuator saturation. The dispersion law could also be promoted to 3D for the same effect, by extending the gradient sum to z.
- **Hysteresis on the cover assignment.** The min-cost matching is recomputed every tick on the latest poses, which makes it react instantly to seeker motion but also occasionally swaps the roles of two hiders when their costs cross. A small hysteresis (refuse to switch unless the alternative is better by $\delta > 0$) would damp the swaps without changing the steady-state assignment.
- **Connectivity-preserving dispersion.** The current Coulomb law diverges in the limit of zero distance and decays in the limit of infinite distance, which makes the swarm spread to the boundary. A connectivity-preserving alternative (e.g. a potential with a finite well at a target spacing) would let the swarm stay inside a sensing radius for explicit position broadcasts, matching the wireless-range constraint of the real Volìere hardware.

A File listing

File	LoC (approx.)	Purpose
`cf2_milestone1.py`	~ 200	Main hider controller.
`cf2_sensing.py`	~ 150	FoV sensing primitives.
`cf2_consensus.py`	~ 60	Obstacle consensus law.
`cf2_hider.py`	~ 130	Inter-hider comms + cover geometry.
`rmtt_seeker.py`	~ 200	Seeker controller + `can_see`.
`game_referee.py`	~ 60	Catch detection.
`sim_viz.py`	~ 170	Cones, trails, HUD.
`tp_algos.py`	~ 360	Required entry-point shim plus controller-level 3D drone-to-drone repulsion.
`standalone_sim.py`	~ 420	The simulator (provided).